Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS132 - Imperative Programming

Practical 1 Specifications

Release Date: 02-03-2026 at 06:00

Due Date: 06-03-2026 at 23:59

Late Deadline: 08-03-2026 at 23:59

Total Marks: 100

Read the entire specification before starting with the practical.

Contents

1 General Instructions **3**

2 Overview **3**

3 Background **3**

4 Your Task **5**

4.1 InputOutput 5

4.1.1 Functions 5

4.2 Receipt 8

4.2.1 Functions 8

4.3 SpotifyInterface 11

4.3.1 Members 11

4.3.2 Functions 11

5 Implementation Details **13**

6 Getting Started **14**

7 Upload Checklist **14**

8 Submission **15**

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- This assignment should be completed individually.
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 48 hours after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (e.g. the presence or absence of certain functions).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of C++11 or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you submit will be overwritten during marking to ensure compliance. **Please ensure you use C++98.**
- All functions should be implemented in the corresponding `.cpp` file. No inline implementation in the header file apart from the provided helper functions.
- The usage of ChatGPT and other AI-related software to generate submitted code is strictly forbidden and will be considered plagiarism.

2 Overview

In this practical, you will work with string variables to model a Spotify-style listening receipt. You will practise separating concerns across multiple files, reading user input, storing values in variables, and printing formatted output to the terminal.

3 Background

Spotify Wrapped is an annual feature that gives users a personalised summary of their listening habits. At its core, each receipt is simply a collection of data fields: who made the song, what it is called, how many times you played it, and how long it is.

In this practical, you will build a simplified version of a Spotify receipt. Each receipt stores six pieces of information about a single song, joined together into one string using the pipe character `|` as a delimiter:

- **Artist Name**
- **Song Title**
- **Album Name**
- **Play Count** – how many times the song was played
- **Song Duration** – duration of the song in seconds
- **Genre**

Consider the following information:

- Artist Name: TheWeeknd
- Song Title: BlindingLights
- Album Name: AfterHours
- Play Count: 312
- Song Duration: 200.5
- Genre: Pop

Using the functions you will develop in this practical, this information can be expressed as a receipt string:

```
TheWeeknd|BlindingLights|AfterHours|312|200.5|Pop
```

Which can then be extracted to give us back each individual field:

- Artist Name: TheWeeknd
- Song Title: BlindingLights
- Album Name: AfterHours
- Play Count: 312
- Song Duration: 200.5
- Genre: Pop

4 Your Task

You are required to implement all of the functions laid out in this section. You have been supplied with the completed **Header** files and partially completed **Implementation** files. See Section 7 for the names of the files you need to upload.

Task	Marks
1 InputOutput	18
2 ReceiptExtractors[1]	27
3 ReceiptForm[2]	9
4 ReceiptPrint[3]	20
5 SpotifyInterface[4]	16
6 FormReceiptFromUserInput[5]	10

Table 1: Mark allocation

Note on functions: In Logo you used *commands* and *reporters*. In C++ these are called **functions**. A function that returns a value (like a reporter) is declared as `returnType functionName(parameters)`. A function that does not return a value (like a command) uses `void` as the return type. For example, `std::string askForStringInput(std::string message)` takes in a `string` parameter named `message` and returns a `string`.

4.1 InputOutput

This file is responsible for the generic input and output functions that are used throughout the practical.

4.1.1 Functions

- `void printOut(std::string message)`
  - This function takes in a `string` parameter named `message` and returns nothing (`void`).
  - It should display the `message` to the terminal followed by a newline.
  - Example – calling `printOut("Now Playing")` should produce:

```
Now Playing
```

- `std::string askForStringInput(std::string message)`
  - This function takes in a `string` parameter named `message` and returns a `string`.
  - It should display the `message` to the terminal followed by a newline, read a line of input from the user, and return that input.
  - *Hint: The user's input may contain spaces (e.g. "The Weeknd"). Make sure the full line is captured.*
  - Use the following as a starting point:

```
std::string askForStringInput(std::string message)
{
    /*Place your variable creation code here*/

    /*Place your printout code here*/

    /*Place your getline code here*/

    /*Return the user input here*/
    return
}
```

- `int askForIntInput(std::string message)`
  - This function takes in a `string` parameter named `message` and returns an `int`.
  - It should display the `message` to the terminal followed by a newline, read a line of input from the user, and return it converted to an `int`.
  - *Hint: Read the input as a string, then convert it to an `int` using the provided `strToInt` function.*

```
int askForIntInput(std::string message)
{
    /*Place your variable creation code here*/

    /*Place your printout code here*/

    /*Place your getline code here*/

    /*Convert your string to int using strToInt and return it*/
    return
}
```

- `float askForFloatInput(std::string message)`
  - This function takes in a `string` parameter named `message` and returns a `float`.
  - It should display the `message` to the terminal followed by a newline, read a line of input from the user, and return it converted to a `float`.
  - *Hint: Read the input as a string, then convert it to a `float` using the provided `strToFloat` function.*

```
float askForFloatInput(std::string message)
{
    /*Place your variable creation code here*/

    /*Place your printout code here*/

    /*Place your getline code here*/
```

```
    /*Convert your string to float using strToFloat and return it*/
    return
}
```

4.2 Receipt

This file contains all the functionality required to create and extract data from receipt strings. All six extractor functions return a `string` – the field is not converted to a number.

4.2.1 Functions

- `std::string extractArtistName(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **artist name**.
  - The artist name is the first field (before the first `|`).
  - *Hint: Use the provided `findOccurance` function together with `substr` to extract the field. Documentation for `substr` can be found at `https://cplusplus.com/reference/string/string/substr/`.*
- `std::string extractSongTitle(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **song title**.
  - The song title is the second field (between the 1st and 2nd `|`).
- `std::string extractAlbumName(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **album name**.
  - The album name is the third field (between the 2nd and 3rd `|`).
- `std::string extractPlayCount(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **play count**.
  - The play count is the fourth field (between the 3rd and 4th `|`).
- `std::string extractSongDuration(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **song duration**.
  - The song duration is the fifth field (between the 4th and 5th `|`).
- `std::string extractGenre(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns a `string` containing the **genre**.
  - The genre is the sixth and last field (after the 5th `|`).

- `std::string formReceipt(std::string artistName, std::string songTitle, std::string albumName, std::string playCount, std::string songDuration, std::string genre)`
  - This function takes in six `string` parameters and combines them into a single receipt string using `|` as the delimiter.
  - The fields must appear in the following order: artist name, song title, album name, play count, song duration, genre.
  - *Hint: Build the result by concatenating the fields and the `"|"` separator using the `+` operator.*
  - Using the example from Section 3, the call:

    ```
    formReceipt("TheWeeknd","BlindingLights","AfterHours","312","200.5","Pop")
    ```

    should return:

    `TheWeeknd|BlindingLights|AfterHours|312|200.5|Pop`

- `void prettyPrint(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns nothing (`void`).
  - It prints the receipt in a labelled, human-readable format. Use your extractor functions to retrieve each field.
  - In the format below: `$` indicates a newline, `_` indicates a space, and `<field>` indicates the value that should be printed.

    ```
    ----_SPOTIFY_RECEIPT_----$
    Artist_______:_<artistName>$
    Song_________:_<songTitle>$
    Album________:_<albumName>$
    Play_Count___:_<playCount>$
    Duration_(s)_:_<songDuration>$
    Genre________:_<genre>$
    -------------------------$
    ```

  - Example output using the receipt from Section 3:

    ```
    ---- SPOTIFY RECEIPT ----
    Artist       : TheWeeknd
    Song         : BlindingLights
    Album        : AfterHours
    Play Count   : 312
    Duration (s) : 200.5
    Genre        : Pop
    -------------------------
    ```

- `void summaryPrint(std::string receipt)`
  - This function takes in a `string` parameter named `receipt` and returns nothing (`void`).

- It prints a single-line summary of the receipt. The symbols `$`, `_`, and `<>` mean the same as in `prettyPrint` above.

```
<songTitle>_by_<artistName>_-_played_<playCount>_times$
```

- Example output using the receipt from Section 3:

```
BlindingLights by TheWeeknd - played 312 times
```

4.3 SpotifyInterface

This file acts as the interface between the user and the receipt system. It collects all the information needed to build a receipt string.

4.3.1 Members

- `std::string artistRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the artist name: "
  ```

- `std::string songRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the song title: "
  ```

- `std::string albumRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the album name: "
  ```

- `std::string playCountRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the play count: "
  ```

- `std::string durationRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the song duration in seconds: "
  ```

- `std::string genreRequest` – Initialise in `SpotifyInterface.cpp` as:

  ```
  "Please enter the genre: "
  ```

4.3.2 Functions

- `std::string askUserForArtistName()`
  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for an **artist name** using `artistRequest` and return it.
- `std::string askUserForSongTitle()`

  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for a **song title** using `songRequest` and return it.

- `std::string askUserForAlbumName()`
  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for an **album name** using `albumRequest` and return it.

- `std::string askUserForPlayCount()`
  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for a **play count** using `playCountRequest` and return it.

- `std::string askUserForSongDuration()`
  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for a **song duration** using `durationRequest` and return it.

- `std::string askUserForGenre()`
  - This function takes no parameters and returns a `string`.
  - Using the functions already implemented in `InputOutput`, prompt the user for a **genre** using `genreRequest` and return it.

- `std::string formReceiptFromUserInput()`
  - This function takes no parameters and returns a `string`.
  - It should prompt the user for all six fields in the following order: **artist name**, **song title**, **album name**, **play count**, **song duration**, **genre** – using the `askUserFor*` functions above.
  - Use `formReceipt` to combine the collected values into a receipt string and return it.
  - Using the example from Section 3, the returned result should be:

    `TheWeeknd|BlindingLights|AfterHours|312|200.5|Pop`

  - *Hint: It is recommended that you draw a flowchart or use pseudocode to plan out this function before implementing it. Tutors will only assist with this function if you have drawn up a flowchart or equivalent plan.*

5 Implementation Details

- You must implement the functions exactly as declared in the header files. Failure to do so will result in compilation errors on FitchFork.
- You may only use **C++98**.
- Do not add `using namespace std` to any header file.
- You may only use the following libraries (already included in the provided header files):
  - `string`
  - `iostream`
  - `cstdlib`
  - `cstdio`
  - `sstream`
- You are supplied with a `main.cpp` to help you compile and test your code as you go:

```cpp
#include <iostream>
#include "InputOutput.h"
#include "Receipt.h"
#include "SpotifyInterface.h"

using namespace std;

int main()
{
    cout << "Given Example" << endl;
    string receipt = formReceipt("TheWeeknd", "BlindingLights",
                                 "AfterHours", "312", "200.5", "Pop");
```

```
    cout << "Pretty Print" << endl;
    prettyPrint(receipt);
    cout << "Summary Print" << endl;
    summaryPrint(receipt);

    cout << "User Input" << endl;
    receipt = formReceiptFromUserInput();
    cout << "Pretty Print" << endl;
    prettyPrint(receipt);
    cout << "Summary Print" << endl;
    summaryPrint(receipt);
}
```

6 Getting Started

Follow these steps to write, compile, and run your practical:

1. Open the appropriate terminal in the folder you are working in. If you are using WSL, open a WSL terminal in the folder for Practical 1.

2. Edit the `.cpp` files using a text editor (Notepad, VSCode, Notepad++, etc.). **Do not edit the `.h` files.**

3. Use the commands from **SU 2.7** to compile your files. Remember to save before compiling. You can also use the following command directly:

```
g++ -std=c++98 -g *.cpp -o main
```

4. Run your program with:

```
./main
```

5. Once all functions are complete, create a zip file containing all of your `.cpp` files and submit it to FitchFork.

7 Upload Checklist

Place the following files into a zip archive named `uXXXXXXXX.zip` where `XXXXXXXX` is your student number:

- `InputOutput.cpp`
- `Receipt.cpp`
- `SpotifyInterface.cpp`
- `main.cpp`

The files must be in the **root** of the zip – when you open the archive you should see the files immediately, not inside another folder. If you are unsure how to create an archive, review Tutorial 3's slides.

8 Submission

Submit your zip file on the FitchFork website (`https://ff.cs.up.ac.za/`). All functions must be implemented (or at least stubbed so the code compiles) before submission.

Your `.h` files will be overwritten during marking – do not alter them.

You have **10 submissions** and your final submission's mark will be your grade. Upload to the Practical 1 slot on FitchFork before the deadline. **No late submissions will be accepted!**